

Complexity Analysis

Complexity analysis is the theoretical evaluation of how much resources an algorithm consumes as the input size (n) grows

In it we want to achieve **platform-independence**:

Use of an abstract machine that uses steps of time & units of memory instead of seconds or bytes.

- each **elementary operation** takes 1 step
- each **elementary instance** ^{copy} takes 1 unit of memory

TWO dimensions of complexity:

- 1) Time complexity
- 2) Space complexity

Time complexity:

Standard Analysis techniques:

- 1) Constant time statements
- 2) Analyzing loops
- 3) Analyzing nested loops
- 4) Analyzing sequence of statements
- 5) Analyzing Conditional Statement.

Now we will start ~~solving~~ finding complexities of algorithms ~~but~~ & in them we will find a cert function & ~~might~~ Big-Oh (worst case).

1) Analyzing an Algorithm:

consider question:

```
int sum ( int A[], int N) {
```

```
    int s = 0;
```

```
    for (int i = 0; i < N; i++)
```

```
        s = s + A[i];
```

```
    return s;
```

```
}
```

Now to find its cost function $f(n)$ we need to look at all of its operations.

```
int sum ( int A[], int N) {
```

```
    int s = 0; — once
```

```
    for (int i = 0; i < N; i++)
```

```
        s = s + A[i]
```

N times N times N+1 times N times (reference).

```
    return s; — once
```

```
}
```

so label:

$$f(n) = 5N + 4.$$

why did condition $i < N$ run $N+1$ times?

when loop runs, it runs & condition fulfills N times but fails 1 time but in all this condition is checked $\frac{N}{1} + \frac{1}{1}$ times.

Success attempts

fail attempts

all other things inside loop & it will run N time because they only run when condition is true.

for big-O : ignore constants & so

$O(N)$

so for big-O notation we can even do it quickly without finding cost function because it ignore most details.

Imp points:

- Declaration alone (int j) is considered of O cost. Assignment operation has 1 cost.

2) Analyzing Simple loops:

Every loop has 2 parts:

- 1) How many iterations are performed?
- 2) How many steps per iteration?

e.g.

```
int sum = 0, j;
```

```
for (j = 0; j < N; j++)
```

```
    sum = sum + j;
```

2 steps.

- ~~loop~~ iterations = N .

- steps per iteration = $2 = O(1)$.

in big-O

so $N * O(1) = O(N)$.

Q.

```

int sum = 0;
for (i = 1; i < n; i = i + 2)
    sum = sum + i;

```

2 steps.

iterations = $N/2$
 # of steps per iteration = 2 = $O(1)$

$$\frac{N}{2} * O(1) = O\left(\frac{N}{2}\right) = O(N)$$

constants ignored.

Question:

```

int sum = 0
for (i = 1; i < 100; i = i + 1)
    sum = sum + i;

```

~~100~~ $100 * O(1) = O(100) = O(1)$

3) Analyzing Nested Loops:

For this, you should treat each loop like a single loop & evaluate each level of nesting as needed.

- In this typically ignore, any body of a loop for cost function & we only include:

- Conditional & of loop
- the inner most body steps.

As we go down the loops in nesting we need to multiply its own complexity with the amount of time its previous loops are calling it.

see example on next page.

Example 1:

```

int j, k;
for (j=0; j<n; j++)
    for (k=0; k<n; k++)
        sum += k+j;

```

$n+1$ times.
 $n * (n+1)$ times
 own conditional statement complexity
 The outer loop runs its body n times so multiplies it here.
 n
 The loop body runs n times
 from outer loop.

$$f(n) = (n+1) + (n * (n+1)) + (n * n)$$

$$= n+1 + n^2 + n + n^2$$

$$F(n) = 2n^2 + 2n + 1$$

$$F(n) = O(n^2).$$

as the loop nesting increases number of multipliers of a statement will also increase.

Example 2:

```

for (i=0; i < n; i++) {
    for (j=0; j < n; j++) {

```

$c[i, j] = 0$; $\rightarrow$ ignored as only inner most body is considered only.

```

        for (k=0; k < n; k++) {

```

```

             $c[i, j] = c[i, j] + A[i, k] + B[j, k];$ 
        }
    }
}

```

$n * n * n$

$$F(n) = (n+1) + (n * (n+1)) + (n * n * (n+1)) + n^3$$

$$= n+1 + n^2 + n + n^3 + n^2 + n^3$$

$$F(n) = 2n^3 + 2n^2 + 2n + 1$$

$$F(n) = O(n^3)$$

Example 3:

```
for (j=0; j<N; j++) {  
    for (k=N; k>0; k--) {  
          
    }  
}
```

doesn't matter step up or down
remains same so
 $O(N^2)$.

Example 4:

```

for (i = 1; i <= n; i++) {
    for (j = 1; j <= i; j++) {
        statement;
    }
}

```

For question like these try to do some iterations:

i	j	Number of iterations of j
i = 1	j = 1	j <= i ✓ yes j++
	j = 2	j <= i ✗ No
i = 2	j = 1	j <= i ✓ yes j++
	j = 2	j <= i ✓ yes j++
	j = 3	j <= i ✗ No
i = 3	j = 1	j <= i ✓ yes j++
⋮	⋮	⋮

so by this we understand total number of iterations/steps for the inner loop:

number of runs: 1, 2, 3, 4, 5, ...

i=1 i=2 i=3 i=4 i=5

Now if we see this with basis of n , it becomes a sum:

$$1 + 2 + 3 + 4 + \dots + N$$

sum of arithmetic series:

$$\text{so } O\left(\frac{N * (N+1)}{2}\right) = O(n^2)$$

so for these Questions if we can find a series then that sum of series allows us to find the complexity.

4) Analyze when Index is not linear:

Example 1:

```
i = 1  
while (i < n) {  
    i = 2 * i;  
}
```

we need to see ~~to~~ & find a pattern/sense to understand these problems.

first make a table of iterations:

~~i = 1~~
~~i = 1~~ i = 1 ; i < n ;
i = 2 ; i < n ;
i = 4 ; i < n ;
i = 8 ; i < n ;
i = 16 ; i < n ;
i = 32

so pattern followed is:

$$2^0, 2^1, 2^2, 2^3, 2^4, \dots$$
$$i = 2^k$$

2 steps)

- 1) Find how is i changing (already done)
- 2) See the stopping condition.

in code it stops when
 $i \geq n$

so

~~2~~

$$2^k = n \quad \# \text{ as } i \geq n \text{ so } i = 2^k$$

$$k = \log_2(n).$$

$$F(n) = O(\log(n)).$$

Example 2:

$$p = 0;$$

for ($i = 0$; $p < n$; $i++$) {

$$p = p + i;$$

}

for such questions :

1) Find i & p relationship at the end with n as the number of iterations.

2) Then seeing that we ~~know~~ know $p < n$, so by equation p in terms of i we get i in relation with n & we can calculate complexity.

$i = 1$	$p = 1$	$(0 + 1)$
$i = 2$	$p = 3$	$(0 + 1 + 2)$
$i = 3$	$p = 6$	$(0 + 1 + 2 + 3)$
$i = 4$	$p = 10$	$(0 + 1 + 2 + 3 + 4)$

day/date

so

$$p = \frac{i(i+1)}{2}$$

now

~~$p = \frac{i(i+1)}{2}$~~

in Big-O we ignore constants

so

$$p = i^2$$

This is to make it easy.

now substitute i into condition

$$p < n$$

$$i^2 < n$$

$$i < \sqrt{n}$$

so

$$F(n) = O(\sqrt{n})$$

48) Analyzing sequence of Statements:

Multiple different statements together.

In this we consider the complexity of the **worst statement**:

Steps:

- Compute complexity functions individually & add them up.

Example 1:

```
for ( j=0; j < N; j++)  
    for ( k=0; k < j; k++)  
        sum = sum + j * k; }  $O(n^2)$   
+
```

```
for ( i=0; i < N; i++)  
    sum = sum - i; }  $O(n)$   
+
```

```
cout << " Sum=" << sum; }  $O(1)$ 
```

$$F(n) = O(n^2) + O(n) + O(1) \\ = O(n^2)$$

5) Analyzing Conditional Statements:

E.g.

```

if (condition) {
    statement 1; =  $O(n)$ 
}
else {
    statement 2; =  $O(n^3)$ 
}

```

now from this we use **worst case complexity**, so as statement 2 has a more worse complexity ($O(n^3)$) it's complexity is $O(n^3)$

$$P(n) = O(n^3).$$

* But consider an edge case like

```

if (1) {
    statement; =  $O(n)$ 
}

```

always true

Now we know it will always run so hence complexity of else or else if, it doesn't matter. $P(n) = O(n)$ always.

BHATTI

Asymptotic Analysis:

Approximation of an algo using **very large values of N** .

- * Analyze problems of larger input size
- * Consider only leading term of formula
- * Ignore the coefficient of leading term.

we will learn 3 notations:

- 1) Big-Oh Notation
- 2) Omega Notation
- 3) Theta Notation.

• Also remember we will mainly consider growth rate e.g. $2N+3$ will be considered as N when n is very large, in growth rate

1) Big Oh Notation (O) : (Upper bound):

Rules:

- Drop lower order terms.
- Drop constant factors.
- Use smallest possible class of functions.

• This means that as big-oh is upper bound, upper bound can be any upper bound e.g. for $f(n) = 2n$, upper bound can be $O(n)$, $O(n^2)$, $O(n^3)$, $O(n!)$ etc.

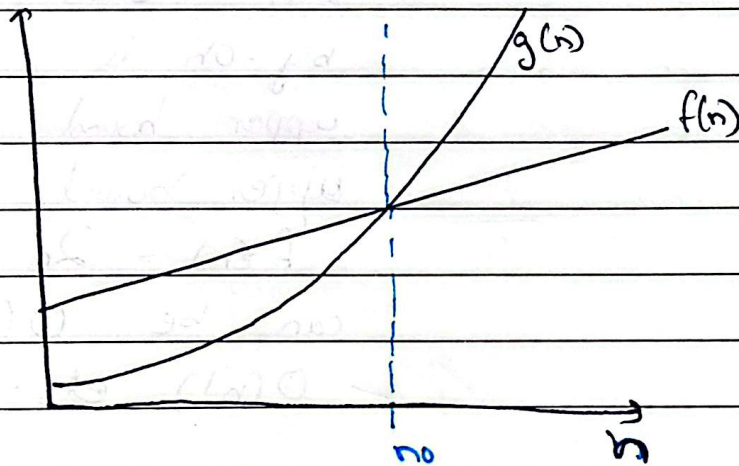
But we will choose the **smallest possible class of functions**. so $O(n)$ for $f(n) = 2n$.

d) ~~Use~~ Use Simplest expression of the class
eg $3n + 5$ is $O(n)$ instead of $O(3n)$.

Formula:

Given 2 functions $f(n)$ & $g(n)$ for input n we say $f(n)$ is in ~~$O(g(n))$~~ $O(g(n))$ if there exists positive constant c & n_0 such that:

$$f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$



* n_0 is the point that defines when $g(n)$ will be bigger than $f(n)$.

Basically we want to find a function $g(n)$ that is eventually always bigger than $f(n)$.

Questions & Examples:

Q1) Prove $2n + 10$ is $O(n)$:

Step 1: (write given).

$$f(n) = 2n + 10$$

$$g(n) = n \neq \Rightarrow O(g(n)) \equiv O(n)$$

$c = \text{constant}$

so $g(n) = n$.

Step 2: (use the big-O equation).

$$f(n) \leq g(n) \cdot c$$

$$2n + 10 \leq n \cdot c \quad \# \text{ substitute values.}$$

Now convert ~~equation~~ in terms of n .

$$10 \leq n \cdot c - 2n$$

$$10 \leq n(c - 2)$$

$$n \geq \frac{10}{c-2}$$

$$\text{so our } n_0 = \frac{10}{c-2}$$

as $n \geq n_0$

BHATTI

Step 3: (Pick a value of c).

remember c is a positive integer
constant. & n_0 is also
positive

$$n_0 = \frac{10}{c-2}, \text{ if we take}$$

$$c = 1 \text{ then } n_0$$

becomes negative
which is wrong.

& we cannot take $c = 2$
because that leads to

$$n_0 = \frac{10}{0}, \text{ undefined.}$$

so

$$\boxed{c > 2}$$

Let's take $c = 3$.

$$n \geq \frac{10}{1}$$

$$n \geq 10$$

Step 4: (Test the values out of big oh)
formula

~~take~~

take $n \geq 10$ & $c = 3$,

for e.g $n = 10$.

$$2(10) + 10 \leq 3(10)$$
$$30 \leq 30$$

for $n = 20$

$$2(20) + 10 \leq 3(20)$$
$$50 \leq 60$$

Hence proved!!!

(Q 2) Prove:

$$f(n) = 10n^2 + 15n \text{ is } O(n^2)$$

$$f(n) = 10n^2 + 15n$$

$$g(n) = n^2$$

$$c = ?$$

$$f(n) \leq g(n) \cdot c$$

$$10n^2 + 15n \leq cn^2$$

again find in terms of n

$$15n \leq cn^2 - 10n^2$$

$$15n \leq n^2 (c - 10)$$

$$\frac{15}{c-10} \leq \frac{n^2}{n}$$

$$n \geq \frac{15}{c-10}$$

so to take a value of c :

$$c > 10$$

day/date

let's take $c = 15$

$$n \geq \frac{15}{15-10}$$

$$n \geq 3$$

so $n_0 = 3$, which is a positive.

check:

$n=3$:

$$10(3)^2 + 15(3) \leq 15 \cdot (3)^2$$

$$90 + 45 \leq 135$$

$$135 \leq 135 //$$

TRUE

so

Proved!!!

IMP:

* ~~Basically~~ Basically what we need to do is find a value of "c" & n_0 that allow the function to be true.

Another Method: (much better & easier):

Q: Prove $10n^2 + 15n + 10$ is $O(n^2)$
~~for $n \geq 10$~~

$$f(n) = 10n^2 + 15n + 10$$

$$g(n) = n^2$$

$$C = ?$$

$$f(n) \leq c \cdot g(n)$$

$$10n^2 + 15n + 10 \leq cn^2$$

establish a lower bound on n , so $n \geq 1$

now we know:

$$10n^2 \leq cn^2$$

$$\text{if } c \geq 10$$

$$15n \leq cn^2$$

$$10 \leq cn^2$$

These should always be true.

So simply take the coefficients & add them up:

$$10n^2 + 15n + 10 \leq 10n^2 + 15n^2 + 10n^2$$

$$10n^2 + 15n + 10 \leq (10 + 15 + 10)n^2$$

$$f(n) \leq 35n^2 \quad \text{for all } n \geq 1$$

$$\text{so } c = 35$$

we need one value of c & that is what we got

~~we need one value of c & that is what we got~~

now we also know that

$$n \geq n_0$$

&

$$n \geq 1$$

so $n_0 \geq 1$, then $C = 35$

so just test it & verify.

Imp:

* ~~test~~ In this way we establish a value of ~~test~~ n_0 before like $n \geq 1$.

* Use all coefficients except if some is negative. If any is negative ignore that.

e.g

$$10N^2 - 10N + 15 \leq C \cdot n^2$$

$$10N^2 - 10N + 15 \leq (10+15)n^2$$

ignored.

* all steps for this way:

1) Pick $n_0 = 1$

2) Set C as sum of all
positive coefficients

3) Since $f(n) \leq C \cdot n^2$ holds for all
 $n \geq 1$ then $f(n) = O(n^2)$.

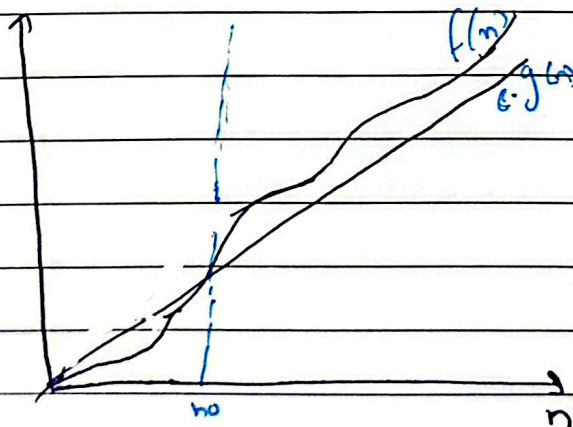
2) Omega Notation (Ω): (lower bound).

Formula:

$f(n) = \Omega(g(n))$ - if there exists **positive constants** $c > 0$ & $n_0 \geq 1$ such that:

$$f(n) \geq c \cdot g(n) \quad \text{for all } n \geq n_0$$

In this we look for the lower bound ($\geq$).
we want to show $f(n)$ is at least
as large as $c \cdot g(n)$.



Examples:

Q) Prove $2n + 3 = \Omega(n)$

Step 1:

$$f(n) = 2n + 3$$

$$g(n) = n$$

$$c = ?$$

Step 2:

$$f(n) > c \cdot g(n)$$

$$2n + 3 \geq c \cdot n$$

Step 3: (make n subject)

$$n(2 - c) \geq -3$$

$$n \geq \frac{-3}{2 - c}$$

Step 4: (choosing a value for c)

now remember n_0 has to ≥ 1 .

we have a negative number
on numerator so we need negative
on denominator

day/date

c can be 9.

$$n \geq \frac{-3}{-1}$$

$$n \geq 3$$

c can be 0.

$$n \geq \frac{-3}{(2-2)}$$

$$n(0) \geq -3$$

$$0 \geq -3 \quad \checkmark$$

This is a
universal
truth

so proved.

Another way to do it:

(Method 2):

Q: Prove $3n^2 + 8n + 5$ is $\Omega(n^2)$

Step 1:

$$f(n) = 3n^2 + 8n + 5$$

$$g(n) = n^2$$

$$c = ?$$

Step 2:

$$3n^2 + 8n + 5 \geq n^2 \cdot c$$

Step 3:

how we know $n \geq 1$ & so our
 $8n > 0$ & $5 > 0$, so we
 can drop our positive terms.
 for finding c .

$$3n^2 + 8n + 5 \geq 3n^2 + 0 + 0$$

only this coefficient is dropped

so our $n \geq 1$ & $c = 3$.

This is a much easier way.

Q: $5n^2 - 12n$ is $\Omega(n^2)$

now this has a negative value.
in big oh we ignored negatives.
BUT here these are problematic
& should be catered.
because we cannot say.

$5n^2 - 12n > 0$ for $n \geq 1$

This is wrong.

Step 1:

$$f(n) = 5n^2 - 12n$$

$$g(n) = n^2$$

$$c = ?$$

Step 2:

$$5n^2 - 12n \geq c \cdot n^2$$

day/date

now we need to equate all of our terms. (in terms of n^2)

$$5n^2 \geq 5n^2 =$$

$$-12n \geq -12n^2$$

now
solve it

Just took a value
Myself to make
this right

$$-12n \geq -12n^2$$

$$n \geq 1$$

so

$$5n^2 - 12n \geq 5n^2 - 12n^2$$

$$5n^2 - 12n \geq -7n^2$$

But $c > 0$ so
it cannot be
-7.

so now let's take some
other value of n in $12n - 12n^2$
let take -3 (should be a multiple
of the original coefficient = 12)

$$-12n \geq -3n^2$$

$$n \geq 4$$

BHATTI

So no ~~4~~ now-

$$5n^2 - 12n \geq 5n^2 - 3n^3$$

$$5n^2 - 12n \geq 2n^2$$

So

$$c = 2 \quad \& \quad n \geq 4.$$

so solved.

Imp:

* Positive terms:

Drop them directly to create a smaller lower bound expression for $n \geq 1$.

* negative terms:

Bound the negative term using a factor of the dominant term for sufficient large n .

day/date

Q: Prove $4n \log_2(n) + 3n$ is $\Omega(n \log_2(n))$

$$4n \log_2(n) + 3n \geq n \log(n) \cdot c$$

for all $n \geq 2$.

we are taking $n_0 = 2$ because
 $\log_2(2) = 1$ which makes
it well-defined / positive &
easier to use.

$$4n \log_2(n) + 3n \geq 4n \log(n) + 0$$

↓ ignored.

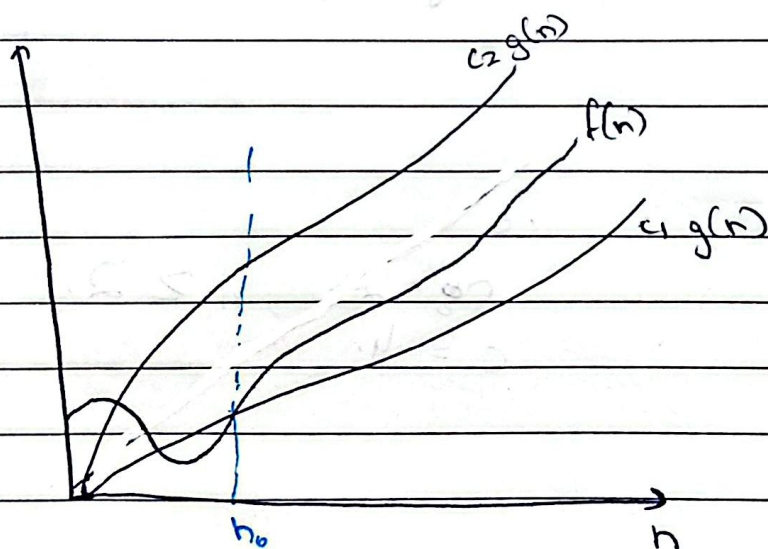
$c = 4$
 $n_0 = 2, n \geq 2$

3) Big Theta Notation (Θ): (Tighter bound):

$\Theta(g(n)) = f(n)$ if there exists
 Positive constants such that $c_1 > 0$, $c_2 > 0$
 & $n_0 > 0$:

$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$

for all $n \geq n_0$.



Just combine omega & Big Oh.
 $\times g(n)$ remains same e.g. n^2 , n^3 etc.

AU 3:Big Oh (O) Ω mega (Ω)Theta (Θ)

• upper bound

• lower bound

• tighter bound

• does not grow any faster than a certain rate

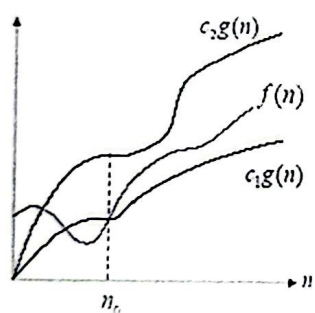
• Grows atleast as fast at a certain rate

• Grows precisely at a certain/same rate.

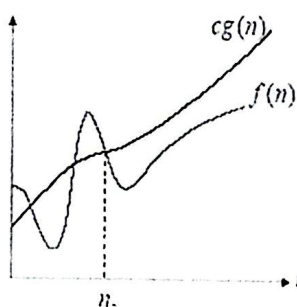
$$f(n) \leq g(n) \cdot c$$

$$f(n) \geq c \cdot g(n)$$

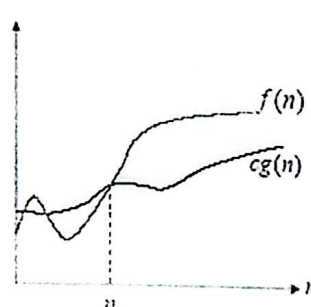
$$c_1 g(n) \leq f(n) \leq c_2 \cdot g(n)$$



$$f(n) = \Theta(g(n))$$



$$f(n) = O(g(n))$$



$$f(n) = \Omega(g(n))$$

Performance Classification:

~~1~~

1

constant complexity

 $\log n$

logarithmic complexity

 n

Linear complexity

 $n \log n$

Linearithmic complexity

 n^2

increases

polynomial complexity

 n^3 (n^b)

polynomial complexity.

 2^n (b^n)

Exponential complexity

 $n!$

Factorial complexity.

Function	Common Name
$N!$	factorial
2^N	Exponential
$N^d, d > 3$	Polynomial
N^3	Cubic
N^2	Quadratic
$N\sqrt{N}$	N Square root N
$N \log N$	$N \log N$
N	Linear
$\sqrt{N}$	Root - n
$\log N$	Logarithmic
1	Constant

Running time grows 'quickly' with more input.

Running time grows 'slowly' with more input.

Properties of big Oh notation:

- 1) Constant factors can be ignored.

$$2n \text{ is } O(n)$$

- 2) Higher powers grow faster.

$$n^r \text{ is } O(n^s) \text{ , } 0 \leq r \leq s$$

e.g

- 3) Fastest growing term dominates a sum

$$\text{e.g } O(n^3) + O(n^2) \text{ then } O(n^3).$$

- 4) Polynomial's growth rate determined by leading term

if f is a polynomial of degree d then f is $O(n^d)$.

5) f is $O(g)$ is **transitive**

if f is $O(g)$ & g is $O(h)$
then f is $O(h)$.

6) **Product of upper bounds is the upper bound of products.**

if f is $O(g)$ & h is $O(r)$
 fh is $O(gr)$.

7) **Exponential functions grow faster than powers.**

n^k is $O(b^n)$ for all
 $b > 1$ & $k \geq 0$

e.g.

n^{20} is 1.05^n

day/date

8) **logarithms grow more slowly than powers.**

$$\log_b n \text{ is } O(n^k) \quad \forall b > 1 \text{ \& } k > 0$$

e.g.

$$\log_2(n) \text{ is } O(n^{0.5}).$$

9) **All logs grow at same rate.**

$$\log_b n \text{ is } O(\log_d n), \quad b, d > 1.$$

~~For~~ e.g. $\log_2 n$ is same as $\log_{10} n$
in growth rate.

ay/date

o -notation & ω -notation:
| small | small:

O -notation & Ω are like $\leq$ and $\geq$
 o -notation & ω are like $<$ and $>$

These are without equality.

o -notation:

$f(n)$ is $o(g(n))$ if for all $c > 0$ there is a constant $n_0 > 0$ such that:

$$f(n) < c g(n) \quad \forall n > n_0$$

~~examples:~~

e.g.: $n^{1.9999} = o(n^2)$

$$n^2 / \lg n = o(n^2)$$

$$n^2 \neq o(n^2) \text{ because } 2 \not< 2 \text{ no equality.}$$

$$\frac{n^2}{1000} \neq o(n^2)$$

ω -notation:

$f(n)$ is $\omega(g(n))$ - for $\exists c > 0$ & $n_0 > 0$
such that:

$$c \cdot g(n) < f(n) \quad \forall n > n_0.$$

examples:

$$\sqrt{n} = \omega(\lg n)$$

$$n^{2.00001} = \omega(n^2)$$

$$n^2 \lg n = \omega(n^2)$$

$$n^2 \neq \omega(n^2)$$

because no
constant.